

MBEYA UNIVERSITY OF SCIENCE AND TECHNOLOGY



COLLEGE OF INFORMATION AND COMMUNICATION TECHNOLOGY

DEPARTMENT OF ELECTRONICS AND TELECOMMUNICATION

ENGINEERING

DESIGN OF INTELLIGENT SOIL ANALYZER AND CROP FERTILIZER

ADVISORY SYSTEM

Student Name	: Projestus K. Protas
Registration Number	: 21100934340029
UE Number	: UE/BETS/24/13663
Course Name	: Project II
Course Code	: ET 8410
Supervisor	: Dr. Juma Ally
Academic Year	: 2024/2025

**A Project Report Submitted in Partial Fulfillment of the
Requirements for Bachelor of Engineering in a
Telecommunication System at Mbeya University of Science and Technology**

Design of Intelligent Soil Analyzer and Crop Fertilizer Advisory System

By

Projestus K. Protas

21100934340029

**Mbeya University of Science and Technology
College of Information and Communication
Department of Electronics and Telecommunication
P.O Box 131, Mbeya**

15th July, 2025

CERTIFICATION

The undersigned hereby certify that the project titled "Design of Intelligent Soil Analyzer and Crop Advisory System" submitted in partial fulfillment of the requirements for the Bachelor of Engineering in Telecommunication Systems at Mbeya University of Science and Technology, has been reviewed and is recommended for acceptance.

Name of the Supervisor: Dr. Juma Ally

Signature of the Supervisor:

COPYRIGHTS

The reproduction, storage, retrieval, or transmission of any part of this project in any form or by any means, electronic, mechanical, photocopying, recording, or otherwise, is strictly prohibited without the prior written permission of Mbeya University of Science and Technology. Unauthorized use or distribution of this project, in whole or in part, is a violation of copyright law and may result in severe civil and criminal penalties. To obtain permission for any usage beyond personal reading, please contact Mbeya University of Science and Technology directly.

DECLARATION

I am, Projestus Protas, declare that I carried out the work reported in this report for the Department of Electronics and Telecommunication Engineering at MUST, under the supervision of Dr. Juma Ally, I solemnly declare that, to the best of my knowledge, no part of this report has been submitted here or elsewhere in a previous application for award or a degree. All sources of knowledge used have been duly acknowledged.

ACKNOWLEDGEMENT

Different people and institutions have supported the mini accomplishment of this book writing. Special thanks to Almighty God above the earth who created the Universe and all creatures in it, I would like to extend my indispensable gratitude to him who gave me strength, wisdom, health and guidance during the whole time of writing this project. I would like to express my gratitude to the Mbeya University of science and Technology (MUST) for providing me an opportunity for the finding out this project and resource personnel who guided me in writing the field project, and special thanks my external supervisor Dr. Juma Ally for his guidance towards writing this book.

I am also grateful to the Electronic and Telecommunication Department and it's staff members including my Internal Instructor Mr. Curthbert John Karawa, for his valuable guidance, commitments, support and encouragement from the beginning of title proposal to this point.

Furthermore, it would be difficult to mention everyone who supported me during my whole Practical Training but I cannot forget to give thanks to Head of Department Dr. Salawa , Mr. Mwanagosi and other staff members, Classmates, my friends, relatives and my caring and loving parent, for their moral and material support and encouragement, may God grant them a long life.

ABSTRACT:

Agriculture is the backbone of the global economy. Many farmers especially in developing countries like Tanzania faces the same problem, they are not selecting suitable crop to plant in each season and how much nutrients level (Fertilizers) are required in the farm. Always they make decision based on their own understanding and knowledge without data driven decision and solution. This project highlights the significance of the Intelligent Soil Analyzer and Crop Fertilizer Advisory System in the agricultural sector, which plays a vital role in Tanzania's economy. However, crop production and resource management continue to pose challenges for many farmers, which make them to struggle to determine which crops are best suited for their soil and which are most profitable.

This project aims to develop an intelligent system that analyzes soil parameters collected by IoT sensors from the farm. It provides recommendations on which crop to cultivate or which fertilizer to use based on the nutrient content in the soil. The system consists of IoT sensor (7 in 1 NPK Sensor) that collect real-time data on soil composition, moisture, humidity, temperature, rainfall, and nutrient levels. The collected data are preprocessed by microcontroller ESP8266 NodeMcu and then transmitted to the Local computer via Wi-Fi, where machine-learning algorithms (specifically Random Forest) and Database are employed to analyze the data and generate recommendations. Finally, these recommendations are sent to the frontend (Web Dashboard) to be shown by a user or farmer to get his recommendations and advices. These recommendations advise the farmer on the most suitable crop to plant, fertilizer types, and the required amounts based on the crop's needs and soil nutrients.

This system is more effective compared to existing methods, as it not only analyzes the soil but also provides specific guidance on crop selection and fertilization. The system's ability to enhance crop productivity, optimize resource utilization, and promote sustainable agricultural practices ultimately supports food security.

Keywords: Intelligent soil analyzer, Crop fertilizer advisory system, IoT sensor, Machine learning, Local Computer and Sustainable agriculture.

TABLE OF CONTENTS

CERTIFICATION	i
COPYRIGHTS.....	ii
DECLARATION	iii
ACKNOWLEDGEMENT	iv
ABSTRACT:	v
LIST OF FIGURES	ix
LIST OF TABLES.....	x
LIST OF ABBREVIATIONS.....	xi
CHAPTER 1. INTRODUCTION	1
1.1 Background.....	1
1.2 Problem Statement.....	2
1.3 Project objective	2
1.3.1 Main objective	2
1.3.2 Specific objectives	2
1.4 Significant of the Project	3
1.5 Scope of the Project	3
1.6 Project Limitation	3
1.7 Project Structure	4
CHAPTER 2: LITERATURE REVIEW	5
2.1 Introduction.....	5
2.2 Related Works.....	5
2.2.1 Crop and Fertilizer Recommendation Systems	5
2.2.2 Machine Learning for Soil Analysis.....	5
2.2.3 Latest Machine Learning Methods	6
2.2.3.1 Random Forest Algorithm	6

2.2.4 Empirical Review	7
2.2.5 Existing Systems.....	8
2.3 Conclusion	9
CHAPTER 3: METHODOLOGY	10
3.1 Introduction.....	10
3.2 System Design Overview.....	10
3.2.1 Working Principle of the System.....	10
3.2.2 Proposed System.....	10
3.3 Components and Requirements.	11
3.3.1 Hardware Components.	11
3.3.2 Software Components.....	12
3.4 Data Collection Method.....	12
3.4.1 Data Collection	12
3.4.2 Datasets Description	13
3.5 Data Processing and Analysis.....	14
3.5.1 Data Processing and Analysis Techniques	14
3.5.2 Flow chat of the system	14
3.5.3 Performance and Evolution of the Model.....	15
3.6 Testing and Simulations.....	16
3.7 Summary.....	18
CHAPTER 4: SYSTEM DESIGN AND IMPLEMENTATION	19
4.1 Introduction.....	19
4.2 System Architecture.....	19
4.2.1 Hardware Components	19
4.2.2 Software Components.....	21
4.3 Summary	22

CHAPTER 5: TESTING AND DISCUSSION	23
5.1 Introduction.....	23
5.2 Testing Methodology.....	23
5.2.1 Units Testing.....	23
5.2.2 System and Functional Testing.....	23
5.3 Results.....	25
5.3.1 Sensor Accuracy and Calibration	25
5.3.2 Testing Examples and its Results.	25
5.4 Discussion of Results and Distinguishing Features.....	28
5.5 Summary	28
CHAPTER 6: CONCLUSION AND RECOMMENDATIONS	29
6.1 Conclusion	29
6.2 Recommendations.....	29
REFERENCES	31
APPENDIX A. SYSTEM SOFTWARE DOCUMENTATION	33
APPENDIX B. SYSTEM SOURCE CODES	38

LIST OF FIGURES

Figure 2. 1:Random Forest Model.	6
Figure 2. 2:Block diagram of the Existing System	8
Figure 1: Block diagram of the proposed system (source: own)	10
Figure 2. Flowchart Diagram (Source: own)	15
Figure 3. Performance of the model (Source :Jupyter Notebook)	16
Figure 4 . Simulation of the circuits.....	17
Figure 5. Simulation of the model.	17
Figure 6. Soil parameter measurement Unit	24
Figure 7. User Dashboard Unit	25
Figure 8. Recommended Crop of soil sample 0001	26
Figure 9. Recommended Crop of soil sample 0002.....	27
Figure 10. Recommended fertilizer of soil sample 0002	27

LIST OF TABLES

Table 1. Fertilizer recommendation Dataset (Source: Kaggle Archives)	13
Table 2. Crop recommendation Dataset (Source: Kaggle Archives).....	14
Table 3. Soil sample and its Values	Error! Bookmark not defined.
Table 4. To show The measured value against the reference values	25
Table 5. Soil parameter of Soil Sample 0001 and its input values	26

LIST OF ABBREVIATIONS

AI	Artificial Intelligences
MUST	Mbeya University of Science and Technology
IoT	Internet of Things
ML	Machine Learning
NPK	Nitrogen Phosphorus Potassium
RF	Random Forest Classifier
TMA	Tanzania Meteorological Authority
TARI	Tanzania Agricultural Research Institute

CHAPTER 1. INTRODUCTION

1.1 Background

Agriculture is a major contributor of the global economy, so this sector always struggles to meet the demand of the population growth, which grows rapidly. Also, agriculture is the backbone of Tanzania's economy, so much so that in 2006, the country adopted a policy called "Kilimo Kwanza," which translates to "Agriculture First." This policy emphasizes the importance of agriculture in the nation's industrial development. Developed countries have greatly benefited from modern technologies in agriculture, which have helped increase both the quality and quantity of crop yields. However, many developing countries, including Tanzania, continue to rely on traditional farming methods. This reliance leads to challenges such as inefficient resource optimization, increased waste, higher costs, climate change impacts, and difficulty in determining which crops are best suited for specific regions.

Most farmers in Tanzania depend on their own understanding and judgment to decide which crops to plant each season. They also often attempt to increase their production by applying various artificial fertilizers, which can eventually lead to environmental harm. While some farmers use soil test kits, these tools are time-consuming and require laboratory facilities, making them less accessible to small-scale farmers.

Therefore, there is a pressing need to develop an Intelligent Soil Analyzer and Crop Fertilizer Advisory System. This system is designed to assist Tanzanian farmers in selecting the correct crops to plant and determining the right amount of nutrients (fertilizers) to apply. The goal is to improve crop production efficiency while preserving soil nutrients for future use.

Soil analyzer is a device that involves understanding the properties and nutrient levels of the soil, enabling farmers to make informed decisions about crop selection and soil management. This process uses various sensors in the field to measure soil parameters such as moisture content, pH levels, temperature, soil moisture and nutrient concentrations. The data collected by these sensors is then transmitted to a central system for processing and analysis using machine learning techniques. Based on the analyzed data, the crop advisory system provides recommendations for the most suitable crops to plant based on specific soil conditions and suggests how much fertilizer is needed.

The development of the Intelligent Soil Analyzer and Crop Fertilizer Advisory System represents a significant advancement in agricultural technology. These systems empower farmers to make data-driven decisions that optimize crop production and promote sustainable farming practices.

1.2 Problem Statement

Farmers in developing countries, particularly in Tanzania, often choose crops to cultivate based on their own understanding and the historical practices of previous farmers. This approach frequently leads to inefficient resource use and low crop yields due to the lack of precise information about soil conditions and the appropriate amount of nutrients needed for the farm. As a result, productivity is reduced, and the environmental impact increases, compromising the soil for future use. Additionally, laboratory soil testing methods are time-consuming, expensive, and not easily accessible to small-scale farmers.

According to **(Motamedi & Villányi, 2024)** currently, soil analyzer systems only suggest suitable crops based on basic soil parameters such as pH and moisture, NPK values. So there is a clear need to develop a real-time Intelligent Soil Analyzer and Crop Fertilizer Advisory System that can provide accurate recommendations and suggestions to farmers about a suitable crop to cultivate in the farm and amount of fertilizer needed basing on nutrients level in the farm. This system should integrate modern technologies such as IoT sensors and machine learning algorithms to assess soil health and recommend the best crop and fertilizer choices. By addressing these challenges, the system aims to improve agricultural productivity and promote sustainable farming practices.

1.3 Project objective

1.3.1 Main objective

To develop an intelligent soil analyzer and crop fertilizer advisory system to enhance agricultural productivity

1.3.2 Specific objectives

- I. To design and implement an IoT-based soil analysis system that can measure key soil parameters such as pH, moisture content, nutrient levels, and temperature in real-time.
- II. To develop Machine-learning algorithms capable of processing soil data and providing accurate crop recommendations based on soil conditions.
- III. To develop a fertilizer recommendation system that identifies the type of crop being cultivated, analyzes soil nutrients, and suggests suitable fertilizers.
- IV. To create a user-friendly dashboard interface that allows farmers to easily access soil analysis reports and crop recommendation results.

1.4 Significant of the Project

The importance of this project lies in its potential to transform agriculture from traditional practices and address critical challenges faced by farmers in selecting suitable crops. The following are key points highlighting the significance of this project:

- i. **Enhanced Agricultural Productivity.**
By providing accurate crop recommendations based on soil conditions, farmers can select the most suitable crops, leading to higher yields and better-quality production.
- ii. **Crop Planning and Crop Rotation Schemes.**
Regular soil analysis helps farmers understand the nutrient status of their soil, allowing them to act appropriately to maintain soil health, resulting in sustained productivity over time.
- iii. **Resource Optimization.**
This system improves the efficient use of fertilizers and water, reducing waste and minimizing costs.
- iv. **Sustainable Agriculture.**
Promoting sustainable farming practices reduces the environmental impact of agriculture by minimizing soil degradation and conserving water and soil nutrient resources.

1.5 Scope of the Project

The design of the Intelligent Soil Analyzer and Crop Advisory System involves collecting soil data such as pH, nitrogen, phosphorus, and potassium levels from the farm field, developing machine-learning algorithms to analyze, process and predict the best crops for specific soil conditions, and determining the appropriate amount of fertilizers and water to use. Additionally, a user-friendly interface will be created for farmers to access the system. The system will be integrated with IoT devices that provide real-time soil data, enhancing the accuracy of the recommendations.

1.6 Project Limitation

However, there are several limitations to consider. The effectiveness of the system depends on the quality and quantity of the collected soil data. Additionally, reliable internet connectivity may be a challenge for some farmers, as internet access is required to view the soil health condition and fertilizers reports on the dashboard.

Finally, convincing farmers, especially those in rural areas, to adopt and trust the technology can be a challenge. Therefore, training on how to use this technology will be necessary to ensure its successful implementation.

1.7 Project Structure

This section of the project report discusses the organization and general structure. The report is divided into six chapters. Also, the chapters are split into multiple subheadings to provide specific information on chapter sections. **Chapter 1** is the introductory part of the project. It provides an introduction to the study and also discusses the project's background. This also outlines the project's objectives, significance, scope, limitations, and structure. **Chapter 2** contains the literature review. It includes a review of several academic publications on the subject. Documents like journals, journals, and publications have been all reviewed. **Chapter 3** shows the methodology. It outlines the project approach or technique to work out the problem. It also sets out the method and dataset used for this project. **Chapter 4** shows the System design and implementation. It outlines all the software and hardware configurations, the dataset processed tools used, the system parameters used, and the evaluation metrics used in this project. **Chapter 5** shows the results and discussion. It contains all the results of each proposed approach. There is also a discussion on the practical aspect of the research and how this work is differentiated in the terms of strength and weakness with another related works. **Chapter 6** is the final chapter of this project. It concludes the study and offers insight into the recommendations and future direction of the project.

CHAPTER 2: LITERATURE REVIEW

2.1 Introduction

This chapter reviews existing studies and related work on soil analysis, crop recommendation systems, and fertilizer advisory technologies, with a focus on the application of machine learning (ML), artificial intelligence (AI), Internet of Things (IoT), and precision agriculture. Precision farming is a technology adopted to enhance crop yields, minimize environmental impacts, and promote sustainable farming practices. Soil analysis and fertilizer recommendation systems play a crucial role in this process. These systems assist farmers in making informed decisions regarding crop selection, nutrient management, and soil health, ultimately leading to optimized agricultural productivity.

2.2 Related Works

2.2.1 Crop and Fertilizer Recommendation Systems

According to **(Csathó et al. 2021)**, various fertilizer recommendation systems used in Hungary were evaluated, focusing on different soil types and crop varieties. Their study emphasizes the need for localized recommendations, noting that the effectiveness of fertilizers often depends on the unique properties of the soil. By comparing multiple systems, they highlight how tailored fertilizer solutions based on soil nutrient profiles can lead to significant improvements in crop yield and resource efficiency. Additionally, **(Abinaya et al. 2024)**

Proposed an intelligent crop recommendation system that utilizes machine learning (ML) and relies on large datasets of soil and environmental conditions. The system uses historical data to predict the best crops for a given location, optimizing decisions based on data trends. Unlike traditional models, this ML-driven approach continuously improves its recommendations as more data becomes available. Machine learning enables the system to adapt dynamically to changing conditions, offering recommendations that are more precise and fostering smarter agricultural practices.

2.2.2 Machine Learning for Soil Analysis

Machine learning (ML) techniques have become fundamental tools for analyzing soil conditions and recommending the most suitable crops and fertilizers. These systems often process complex datasets derived from soil properties, weather conditions, and historical agricultural performance to generate actionable insights. **(Motwani et al. 2022)** focus on the use of ML algorithms for soil analysis. Their system examines key soil characteristics, such as pH, organic matter content, and soil texture, to predict which crops are most likely to succeed in specific soil types. Additionally, **(Akkem et al. 2024)** explore smart farming techniques

powered by AI, where machine-learning models integrate weather, soil, and crop data to recommend fertilizer applications and irrigation schedules. Their approach combines big data analytics with advanced algorithms to optimize farming operations, ensuring the efficient use of inputs such as water and fertilizers.

2.2.3 Latest Machine Learning Methods

Machine learning has significantly advanced agricultural practices by providing data-driven solutions for soil analysis and crop fertilization. After reviewing multiple studies **(Sankareswari & Sujatha, 2024)**, **(Najdenko et al., 2024)** and **(Abinaya et al., 2024)** it is evident that Random Forest and Naive Bayes are algorithms that perform well in both classification and regression tasks. These algorithms are suitable for our project, which requires accuracy in soil fertility analysis and fertilizer recommendations.

2.2.3.1 Random Forest Algorithm

Random Forest (RF) is a machine-learning algorithm that combines multiple models to improve performance and accuracy. The model constructs multiple decision trees and combines their outputs to enhance accuracy and reduce overfitting. Studies have shown that RF achieves high accuracy in quality soil assessment. It is also highly effective with tabular data, even when dealing with missing values. RF handles large datasets, medium-sized datasets, and missing data well, reduces the impact of noise, and identifies important features. These characteristics make it ideal for soil analysis and fertilizer recommendations. Its operations is detailed in figure 2.1

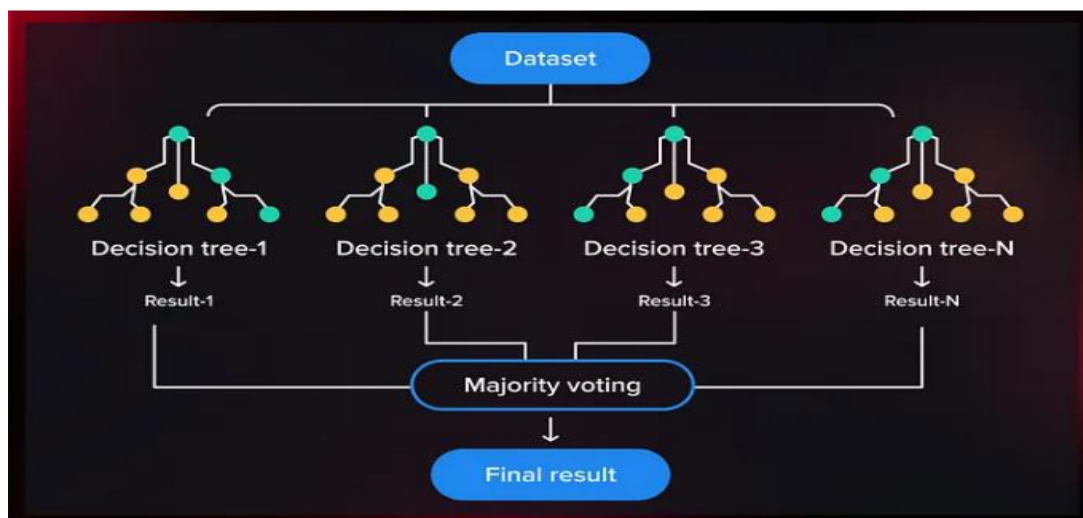


Figure 2. 1.Random Forest Model.

2.2.4 Empirical Review

This section reviews key empirical findings from studies that have implemented projects similar to intelligent soil analysis and crop fertilizer recommendation systems in real-world agricultural settings, exploring their effectiveness, gaps, challenges, and contributions to sustainable farming.

i. Effectiveness

Several studies have investigated the effectiveness of intelligent soil analyzers and crop fertilizer recommendation systems. **(Rajak et al., n.d.)** empirically compared AI-driven fertilizer recommendation systems with traditional methods across various soil types and crops in Hungary. Their findings indicated that AI systems provided more accurate and tailored fertilizer recommendations, accounting for variations in soil properties such as moisture, pH, and nutrient content.

ii. Contributions to Sustainable Farming

Research has also shown the contribution of AI-driven fertilizer recommendation systems to sustainable farming. **(Motwani et al. 2022)** developed a system that recommends fertilizers based on the actual needs of the soil, helping prevent the overuse of chemical fertilizers and thus reducing environmental harm. By improving soil health and optimizing fertilizer usage, the system contributes to more sustainable farming practices and enhanced productivity.

iii. Challenges and Barriers

Despite the contributions to sustainable farming, studies have identified challenges and barriers associated with the implementation of intelligent soil analyzers and crop fertilizer recommendation systems **(Rahman et al. 2024)**, **(Mishra et al. 2021)** and **(Pudumalar et al. 2017)** faced difficulties in accurately measuring nutrient deficiencies in soil, which require advanced testing techniques and specialized equipment. Additionally, a lack of comprehensive soil nutrient datasets challenged the system's ability to predict optimal crops and fertilizers for diverse soil types.

iv. Recommendations for Advanced Technology

Empirical studies have proposed recommendations and future directions for the design and development of advanced intelligent soil analyzers and crop fertilizer recommendation systems. **(A et al. 2022)** suggested incorporating more robust datasets, improving sensor technologies, and integrating machine-learning models that can adapt to diverse environmental and soil conditions to enhance the accuracy and usability of these systems.

2.2.5 Existing Systems

According to (Motamedi & Villányi, 2024) currently, soil analyzer systems only suggest suitable crops based on basic soil parameters such as pH and moisture. However, these systems fail to account for critical factors such as nutrient deficiencies and fertilizer requirements. As a result, farmers may follow the recommended crop choices but experience low yields due to a lack of essential nutrients in the soil. Furthermore, these systems do not provide fertilizer guidelines, leading to nutrient imbalances and potential environmental harm. Figure 2.2 tend to show how the existing system perform.

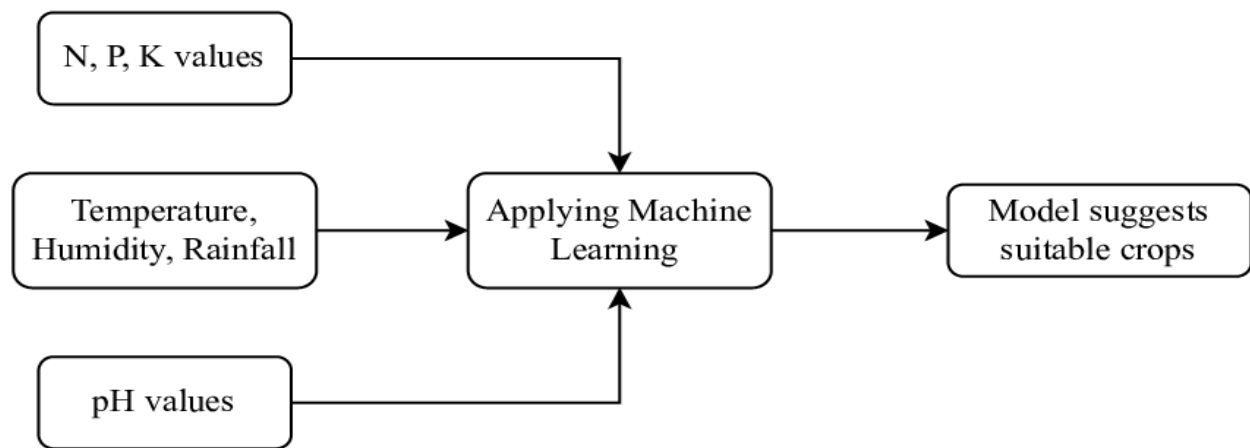


Figure 2. 2. Block diagram of the Existing System

Drawbacks of Existing Systems

1. **No Fertilizer Advisory:** The current system may suggest crops but fails to provide tailored fertilizer recommendations. As a result, users lack guidance on the type, quantity, and timing of fertilizers required for the existing crops.
2. **Lack of Crop Planning:** The system only recommends crops, and farmers tend to choose the same crops each season without considering soil degradation or the benefits of crop rotation. Additionally, there is no integration with existing factors such as the planted crops and available nutrients in the soil.
3. **Nutrient Deficiencies Overlooked:** Farmers proceed to plant the same crops recommended by the system, but the soil may lack essential nutrients, leading to lower yields.

2.3 Conclusion

The integration of AI, machine learning, and IoT technologies into soil analysis and crop recommendation systems paves the way for a more efficient, sustainable, and intelligent agricultural future. From data-driven models that predict crop suitability to real-time soil data and nutrient management, these innovations have the potential to revolutionize farming practices. By incorporating insights from various studies, an intelligent soil analyzer and fertilizer recommendation system can be designed to maximize productivity, optimize resource usage, and promote environmental sustainability.

CHAPTER 3: METHODOLOGY

3.1 Introduction

This chapter presents the methods and approach used in developing and designing the Intelligent Soil Analyzer and Crop Fertilizer Advisory System. It covers an overview of the system, including the hardware and software components, data collection methods, analysis techniques, and the implementation strategy employed to achieve the objectives of this work. In response to the drawbacks identified in the existing systems, this chapter outlines a systematic approach to designing a preferred solution that addresses those shortcomings.

3.2 System Design Overview

The proposed Intelligent Soil Analyzer and Crop Fertilizer Advisory System consists of four core subsystems that work together to achieve our objectives: the soil data conditions devices, data processing and analysis model, advisory system, and user interface.

3.2.1 Working Principle of the System

The system operates by collecting soil data through sensors that measure key soil parameters, including pH, moisture, temperature, and nutrient levels (N, P, K). The collected data are preprocessed by a microcontroller and then transmitted to a remote server for analysis via Wi-Fi. The remote server hosts machine-learning algorithms that analyze soil quality and determine the nutrient requirements of a given crop or suggest the most suitable crop to cultivate. Based on this analysis, the system generates actionable recommendations for farmers, including the type and quantity of fertilizers needed, suitable crop suggestions, and crop rotation schedules.

3.2.2 Proposed System

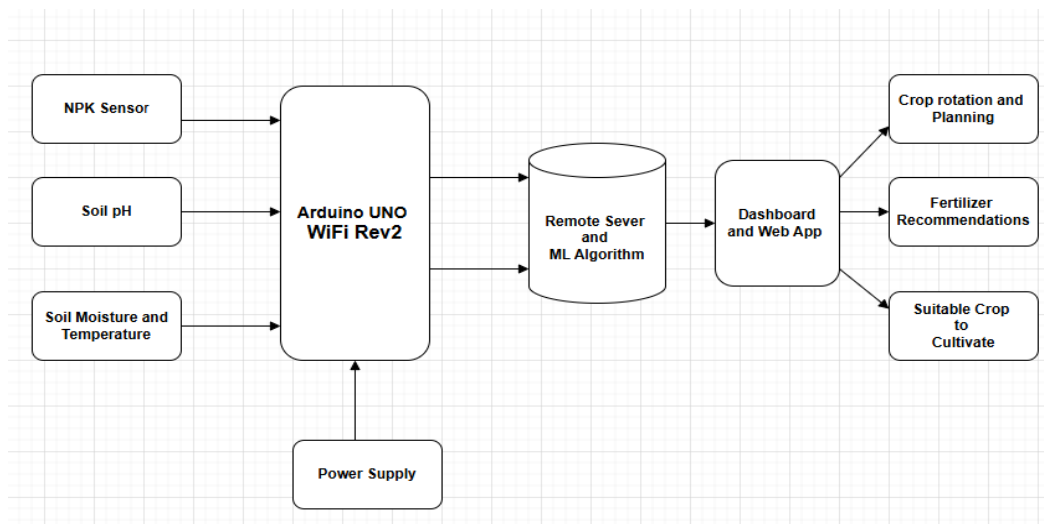


Figure 1: Block diagram of the proposed system (source: own)

3.3 Components and Requirements.

3.3.1 Hardware Components.

1. Computer

The computer used in this project is equipped with an Intel Core i5 CPU, 8GB of RAM, and runs Windows 10 Pro. It handles the system's processing requirements and supports data analysis, machine learning algorithms, and system operations.

2. JXCT Soil NPK Sensor RS485

The JXCT NPK sensor used for soil NPK, Ph., moisture, temperature and humidity measurement with high accuracy, fast response speed and stable output. Long term buried in the soil, resistant to long term electrolysis, corrosion resistance, vacuum potting and waterproof.

Power Supply: 9-12V DC

Output Signal: RS485

Temperature range: -45-115°C

Moisture measuring range: 0-100%RH

Soil pH measuring range: 3-9pH

NPK measuring range: 0-1999mg/kg

This sensor was chosen because is very accurate and comprise seven components in one sensor (7 in 1).

3. Arduino UNO WiFi Rev2

The Arduino UNO WiFi Rev2 is an upgraded version of the classic Arduino UNO, equipped with built-in Wi-Fi and Bluetooth capabilities for preprocessing raw data. This allows seamless integration with IoT projects. The operating voltage is 5V, and the board provides connectivity via the NINA-W102 Wi-Fi and Bluetooth module, enabling remote data transmission to the server for analysis.

This Arduino type was chosen because it was designed with Wi-Fi module, which is used to sends the collected data from the farm to the remote sever for the analysis and processing.

4. Power Supply

Power supply is an electrical device that provide the required voltage and current for specific application or system. In this system the power supply takes an AC input of 240V and convert it to a DC output of 9V for sensor and Arduino. And the conversion process typically involves four stages. LM7809 was used so as to produce constant voltage of 9V.

- a) **Step Down transformer.** This reduce the 240V to a lower AC voltage
- b) **Rectification.** Abridge rectifier convert ac to pulsating DC.
- c) **Filtering.** Series of capacitors smooths the pulsating DC to reduce ripples.
- d) **Voltage Regulation.** A voltage regulator, which is LM7809, ensures a stable 9V DC output.

3.3.2 Software Components.

1. **Arduino IDE:** The Arduino Integrated Development Environment (IDE) is utilized for programming the Arduino UNO WiFi Rev2 microcontroller. It provides an interface to write and upload C++ code, facilitating the development and debugging of the microcontroller's firmware.
2. **Python:** Python is employed for processing and analyzing the data collected from the sensors. Its simplicity and support for built-in libraries such as Pandas, NumPy, and Scikit-learn offer powerful tools for data manipulation and analysis.
3. **User Interface:** A dashboard or web application is developed using Flask, a lightweight Python web framework. Flask's simplicity and ease of integration with machine learning models make it an ideal choice for creating user-friendly interfaces that display data and provide recommendations to farmers. This setup allows for seamless interaction between the user and the underlying machine learning algorithms.

3.4 Data Collection Method

3.4.1 Data Collection

In this project, data was obtained from the farm by soil sensors and the Kaggle platform, a trusted source for varied and high-quality datasets. The dataset includes soil composition parameters such as nitrogen (N), phosphorus (P), potassium (K), pH levels, moisture content, and temperature, along with crop yield data and corresponding fertilizer recommendations. The data was pre-processed to remove inconsistencies, missing values, and outliers to ensure

reliability and accuracy. This dataset serves as the foundation for developing predictive models that analyze soil health and provide optimized fertilizer recommendations to enhance crop productivity. The structured nature of the dataset enables effective training and validation of machine learning algorithms, ensuring precise and data-driven agricultural insights.

3.4.2 Datasets Description

The dataset used for the Intelligent Soil Analyzer and Crop Fertilizers Advisory System consists of two main components: crop recommendation and fertilizer advisory system. The crop recommendation dataset includes 2,200 soil samples with 22 different crop types, each labeled with its optimal growing conditions based on soil properties such as nitrogen (N), phosphorus (P), potassium (K), pH level, moisture content, temperature, and temperature. Additionally, the fertilizer advisory dataset contains 552 soil samples with 11 fertilizer-related variables, providing insights into the best fertilizer combinations for different soil conditions. This well-structured dataset enables the development of an intelligent system capable of providing precise crop selection and fertilizer recommendations, enhancing soil health management and agricultural productivity. This is shown clearly in Table 1 and Table 2.

Table 1. Fertilizer recommendation Dataset (Source: Kaggle Archives)

	Temparature	Humidity	Moisture	Nitrogen	Potassium	Phosphorous
count	552	552	552	552	552	552
mean	28.63044	64.55797	42.84058	28.52174	10.14493	21.11594
std	5.088082	11.88024	11.50728	29.12199	13.45696	14.92051
min	0	50	25	0	0	0
25%	26	54	33	10	0	8
50%	29	62	41	15	0	20
75%	32	68	51	37	18	36
max	38	95	65	126	59	54

Table 2. Crop recommendation Dataset (Source: Kaggle Archives)

	N	P	K	temperature	humidity	ph	rainfall
count	2200	2200	2200	2200	2200	2200	2200
mean	50.55182	53.36273	48.14909	25.61624	71.48178	6.46948	103.4637
std	36.91733	32.98588	50.64793	5.063749	22.26381	0.773938	54.95839
min	0	5	5	8.825675	14.25804	3.504752	20.21127
25%	21	28	20	22.76938	60.26195	5.971693	64.55169
50%	37	51	32	25.59869	80.47315	6.425045	94.86762
75%	84.25	68	49	28.56165	89.94877	6.923643	124.2675
max	140	145	205	43.67549	99.98188	9.935091	298.5601

3.5 Data Processing and Analysis

3.5.1 Data Processing and Analysis Techniques

Data processing and analysis involved cleaning and pre-processing the collected dataset to ensure accuracy and reliability. The dataset, which includes 2,200 soil samples for crop recommendation and 552 samples with fertilizer-related variables, was first checked for missing values, outliers, and inconsistencies. Key soil parameters such as nitrogen (N), phosphorus (P), potassium (K), pH, moisture content, temperature, soil type and crop planted standardized to improve model performance. A Random Forest Classifier was then trained on the processed data to predict the most suitable crop based on given soil conditions. The model analyzes soil characteristics and selects the best-matching crop from the 22 available options. Additionally, for fertilizer recommendations, the model evaluates soil nutrient deficiencies and provides optimized fertilizer suggestions based on the 11 fertilizer variables. The system ensures precision by considering multiple factors, allowing farmers to make informed decisions for better crop yields and efficient fertilizer usage.

3.5.2 Flow chat of the system

The flowchart of the system starts with data collection, followed by pre-processing to clean and normalize the data. The processed data is then used to train a Random Forest Classifier with 10-fold cross-validation. The trained model classifies suitable crops and provides fertilizer recommendations based on soil conditions, which are then presented to the user for optimal agricultural decisions and reports. Then the system continuously measures the soil conditions. This is shown clearly in Figure 2.

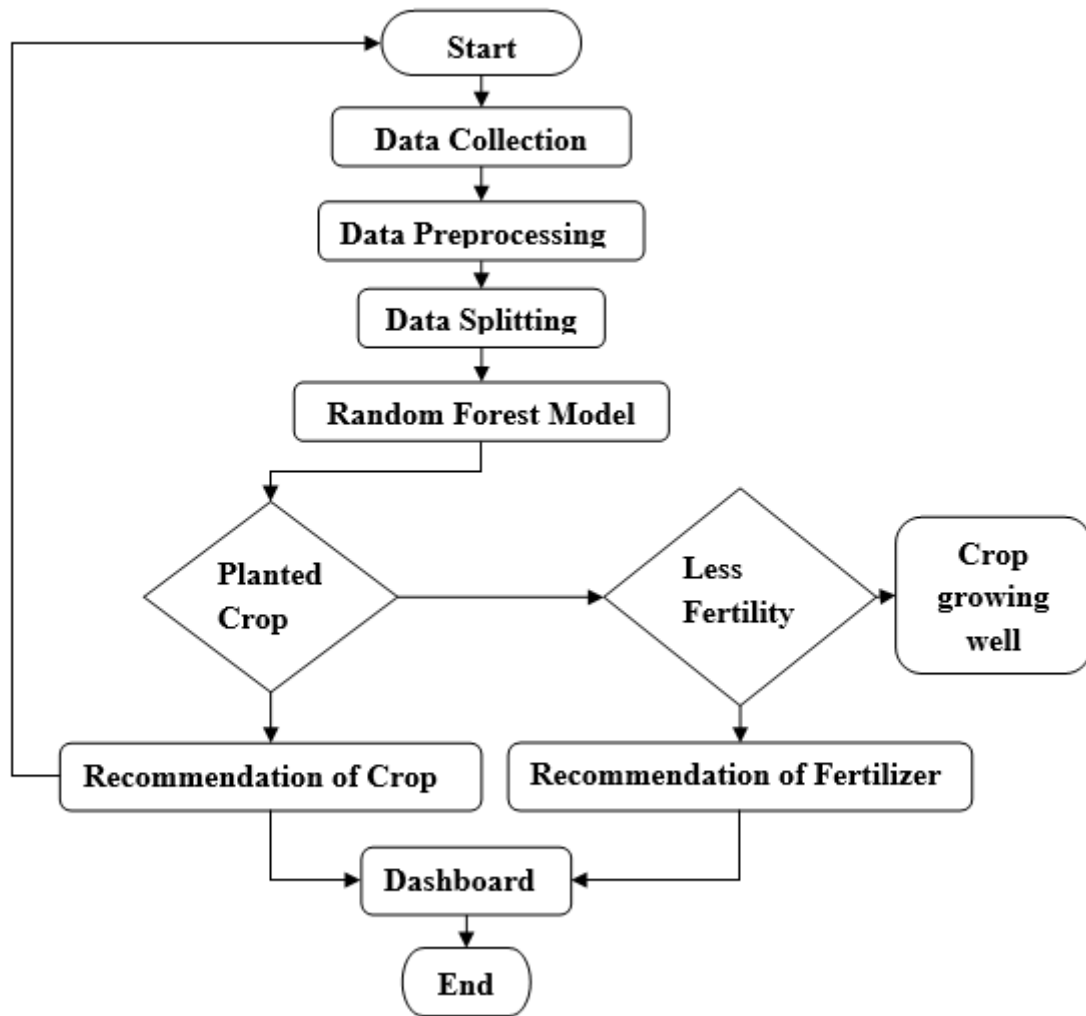


Figure 2. Flowchart Diagram (Source: own)

3.5.3 Performance and Evolution of the Model

The performance of the System was evaluated using cross-validation, a technique used to assess the model's generalization ability and prevent overfitting. Specifically, k-fold cross-validation was applied, where the dataset was divided into k equal subsets (folds). The model was trained on k-1 folds and tested on the remaining fold, repeating this process k times to ensure that every data point was used for both training and validation. The average performance across all folds provided a more reliable estimate of the model's accuracy. This method ensured that the Random Forest Classifier effectively learned from different subsets of data, improving its robustness in predicting suitable crops and recommending optimal fertilizers based on soil characteristics. This is shown clearly in details in Figure 3

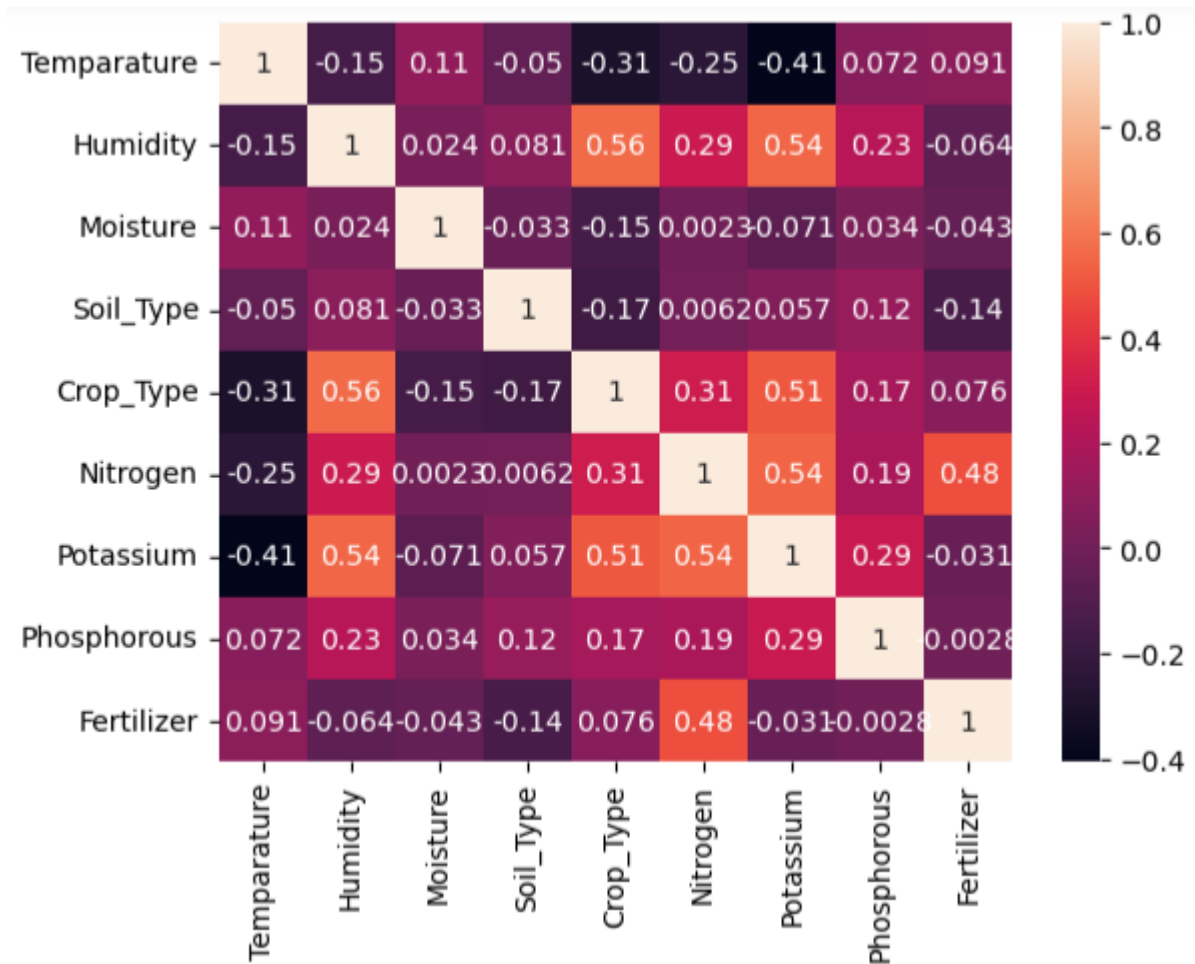


Figure 3. Performance of the model (Source :Jupyter Notebook)

3.6 Testing and Simulations

The system hardware components was designed by different blocks that is Power supply block, Microcontroller module (Arduino microcontroller), sensors component block and virtual terminal to display the collected data from the sensors was simulated in proteus, and the model for recommending crop and fertilizers, python(Jupyter Notebook) was used as simulation tools. The performance and model prediction accuracy was shown clearly in Figure 3.4 and 3.5.

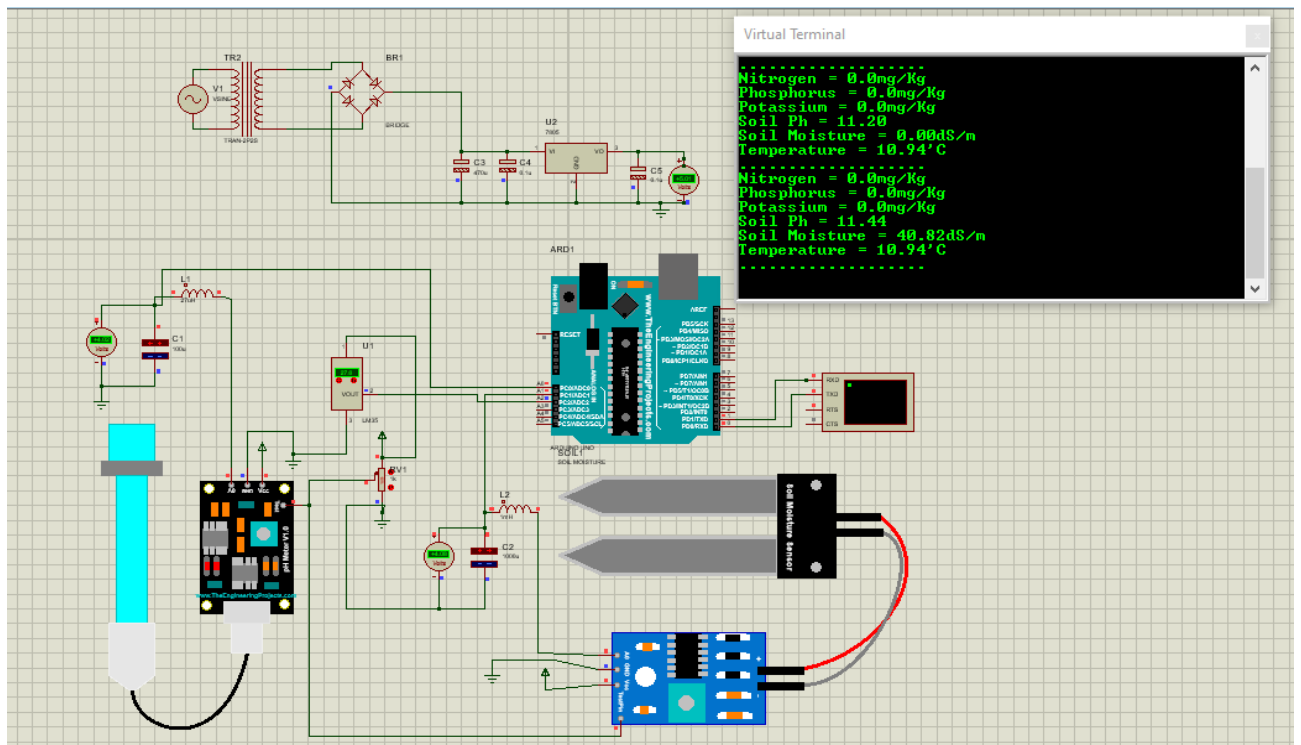


Figure 4 . Simulation of the circuits

Accuracy: 1.0
 Predicted Fertilizer Type: Urea
 Training Accuracy (Fertilizer): 1.0
 Test Accuracy (Fertilizer): 1.0

Crop model accuracy: 0.9931818181818182
 Predicted crop: papaya
 Cross-validation scores (Crop): [0.99772727 0.98636364]
 Average cross-validation score (Crop): 0.99404545

Figure 5. Simulation of the model.

3.7 Summary

In conclusion, this chapter has outlined the detailed design and development of the Intelligent Soil Analyzer and Crop Fertilizer Advisory System. The system uses IoT sensors to collect real-time data on important soil factors like pH, moisture, temperature, and nutrients. The data is then processed and analyzed using machine learning algorithms to provide accurate soil health assessments and personalized recommendations for crop and fertilizer management.

By combining advanced hardware, such as sensors and an Arduino microcontroller, with powerful software tools like Python and Flask, the system offers reliable data analysis and a user-friendly interface. This integrated approach enables farmers to make informed decisions, improving productivity and promoting sustainable farming. The methodology ensures that this system is a valuable step forward in precision agriculture.

CHAPTER 4: SYSTEM DESIGN AND IMPLEMENTATION

4.1 Introduction

This chapter presents a detailed and practical description of the system design and implementation, including hardware configurations, power supply considerations, circuit design, software architecture, and data flow integration.

4.2 System Architecture

The system was designed to provide real-time soil analysis and intelligent recommendations using a combination of hardware and software components. The core architecture consists of:

- NPK sensor and MAX485 module for soil data acquisition
- ESP8266 NodeMCU microcontroller for data transmission
- Local computer for data processing and analysis
- SQLite database for data storage
- Web dashboard for user interaction

These components were integrated to form a closed-loop system that enables efficient data collection, analysis, and farmer-friendly advisory services.

4.2.1 Hardware Components

The hardware forms the tangible interface with the agricultural environment, responsible for capturing the raw data essential for analysis.

4.2.1.1 Power Supply

The entire system requires a stable and reliable power supply. In this project, a 5V DC regulated power supply is used to power both the NPK sensor and the ESP8266 NodeMCU. For field deployment, a rechargeable battery was integrated to ensure continuous operation in remote areas without access to AC source voltages.

4.2.1.2 NPK Sensor and MAX485

A single, multi-parameter NPK sensor was deployed to capture potential soil characteristics. This sensor measures:

- Nitrogen (N) content in soil (mg/kg)
- Phosphorus (P) content in soil (mg/kg)
- Potassium (K) content in soil (mg/kg)
- Soil temperature (°C)
- Soil pH level (scale 1-14)
- Soil moisture content (%)

MAX485 was used as subsystem interface between ESP8266 Node MCU and NPK Sensor, this module was used because the output of NPK Sensor was RS485 and this output cannot be used by microcontroller in the form TTL . That is the reason why Max485 Module was used.

4.2.1.3 ESP8266 NodeMCU microcontroller

The ESP8266 NodeMCU serves as the main data acquisition and transmission unit. It receives digital signals from the NPK sensor through MAX485 Module, preprocesses, calibrate them and transmits the data to the local computer via Wi-Fi.

Advantages of ESP8266 NodeMCU:

- ✓ Built-in Wi-Fi module for easy connectivity
- ✓ Low power consumption
- ✓ Compact and cost-effective design

4.2.1.4 Circuit Diagram

The practical circuit design involves connecting the NPK sensor's output pins to the MAX485 Module and its output pins to input pins on the ESP8266 NodeMCU. The NodeMCU was then connected to the power supply and ESP8266 was configured to transmit data wirelessly to local computer. This is shown clearly in Figure 4. Below

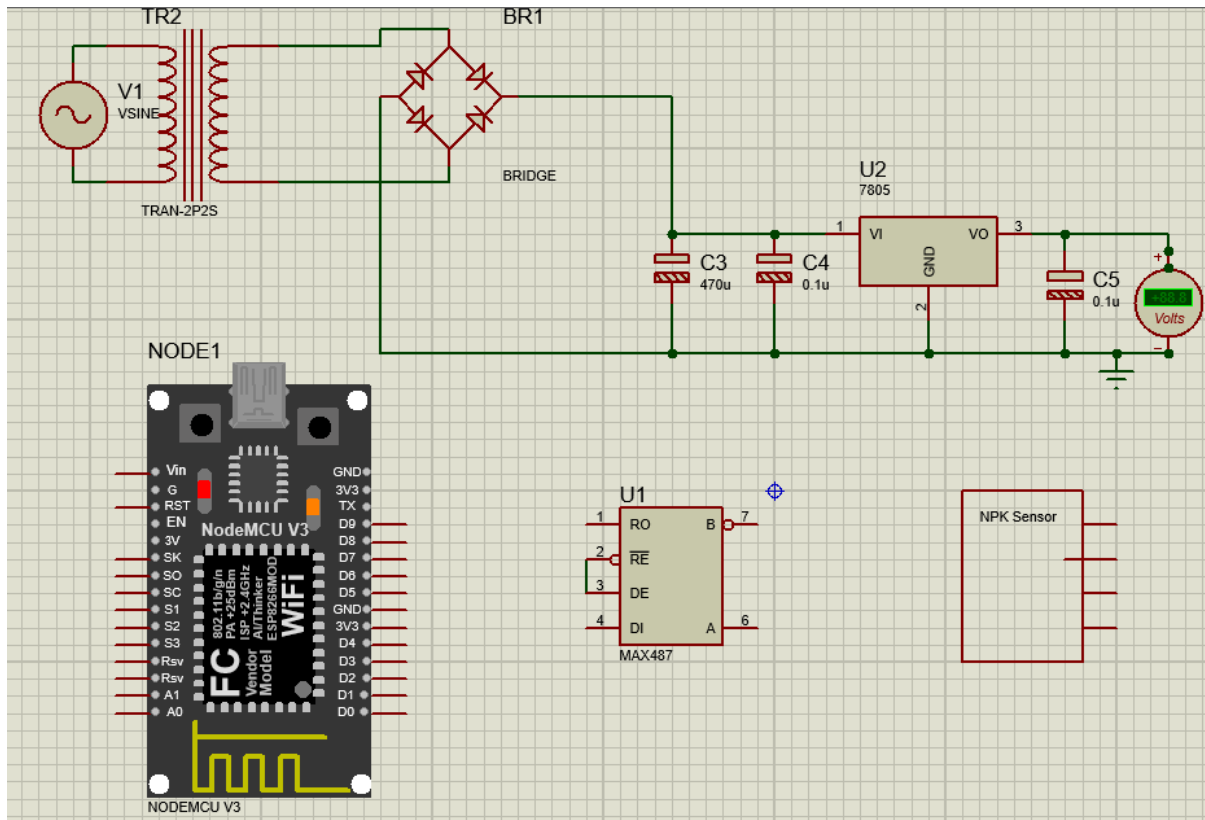


Figure 4. Circuit Diagram of Hardware components

4.2.2 Software Components

4.2.2.1 Local Computer Backend

The local computer acts as the data processing center. It receives transmitted soil data from the NodeMCU and analyzes it using two machine-learning models:

- ✓ Crop Recommendation Model: Determines the most suitable crops based on current soil and environmental conditions.
- ✓ Fertilizer Recommendation Model: Suggests fertilizers required to address specific nutrient deficiencies.

Both models were developed using Random Forest classifiers and trained with datasets from Kaggle. These models were deployed on the local machine and integrated into the backend system using Flask framework.

4.2.2.2 Database

A SQLite database is used to store incoming sensor data, analysis results, and generated recommendations. SQLite was chosen for its lightweight nature and ease of integration with Python and Flask applications.

4.2.2.3 Web Dashboard

The frontend of the system was designed using HTML, CSS, and JavaScript to provide an intuitive, user-friendly interface. The backend was developed using Flask, which handles data routing, model predictions, and database queries.

Features of the dashboard include:

- ✓ Real-time display of soil nutrient data
- ✓ Graphical visualization of trends
- ✓ Personalized crop and fertilizer recommendations

The working integration of software components and data flow is shown clearly in Figure 5.

Below as: -



Figure 5. Working interaction of software components

4.3 Summary

This chapter has provided a comprehensive overview of the "Intelligent Soil Analyzer and Crop Fertilizers Advisory System's" design and implementation. By detailing the architecture of the system, the selection and configuration of hardware and software components. All designed components ensures that each part of the system contributes effectively to the overall goal of empowering farmers with data-driven decision-making capabilities.

CHAPTER 5: TESTING AND DISCUSSION

5.1 Introduction

This chapter presents a comprehensive evaluation of the intelligent soil analyzer and crop fertilizer advisory system through systematic testing. The evaluation encompasses system functionality, accuracy assessment, user acceptance, and practical implementation challenges. The testing methodology follows established agricultural technology validation standards to ensure reliable and actionable results.

5.2 Testing Methodology

5.2.1 Units Testing

- ✓ Unit testing focused on verifying the correct operation of individual modules and components in isolation. This included:
- ✓ Sensor Readout Verification: Testing the NPK sensor's ability to accurately measure N, P, K, temperature, pH, and moisture, and ensuring the ESP8266 NodeMCU correctly reads and formats this data.
- ✓ Wireless Communication Test: Validating the ESP8266's Wi-Fi connectivity and its ability to reliably transmit data to the local computer.
- ✓ Database Integration Test: Confirming that the Flask backend successfully receives, stores, and retrieves data from the SQLite database.
- ✓ Model Loading and Inference Test: Ensuring that the pre-trained Random Forest models (Crop and Fertilizer) load correctly and can generate predictions when provided with sample input data.
- ✓ User Interface Component Test: Verifying the responsiveness and correct rendering of individual HTML, CSS, and JavaScript elements within the user dashboard

5.2.2 System and Functional Testing

System testing evaluated the complete, integrated system against its specified requirements, focusing on overall performance, reliability, and accuracy. This involved:

- ✓ End-to-End Recommendation Accuracy: Assessing the precision of crop and fertilizer recommendations generated by the system based on various simulated soil conditions.
- ✓ System Responsiveness: Measuring the latency from sensor data transmission to the display of recommendations on the user dashboard.
- ✓ Stability and Robustness: Operating the system over extended periods to identify any potential crashes, data corruption, or performance degradation.

The entire system was tested to ensure that each component—from sensor data acquisition to final recommendation—functioned seamlessly. Key aspects validated was:

- ✓ Accurate sensor readings across different soil samples.
- ✓ Reliable wireless data transmission via ESP8266.
- ✓ Correct storage and retrieval of data in the SQLite database.
- ✓ Proper execution of predictive models and generation of recommendations.
- ✓ User dashboard responsiveness and ease of interaction.

Figure 6 and Figure 7 tend to the soil measurement units and user dashboard unit as shown clearly below: -

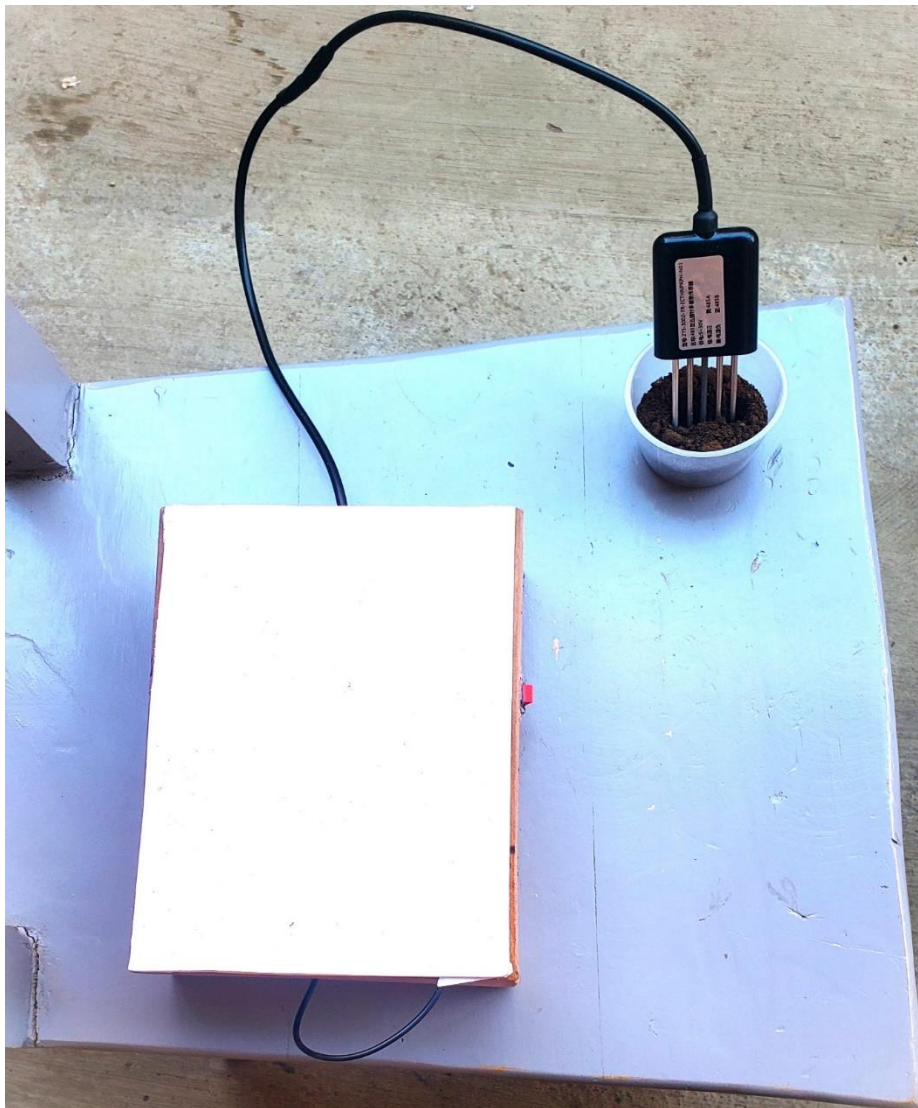


Figure 6. Soil parameter measurement Unit

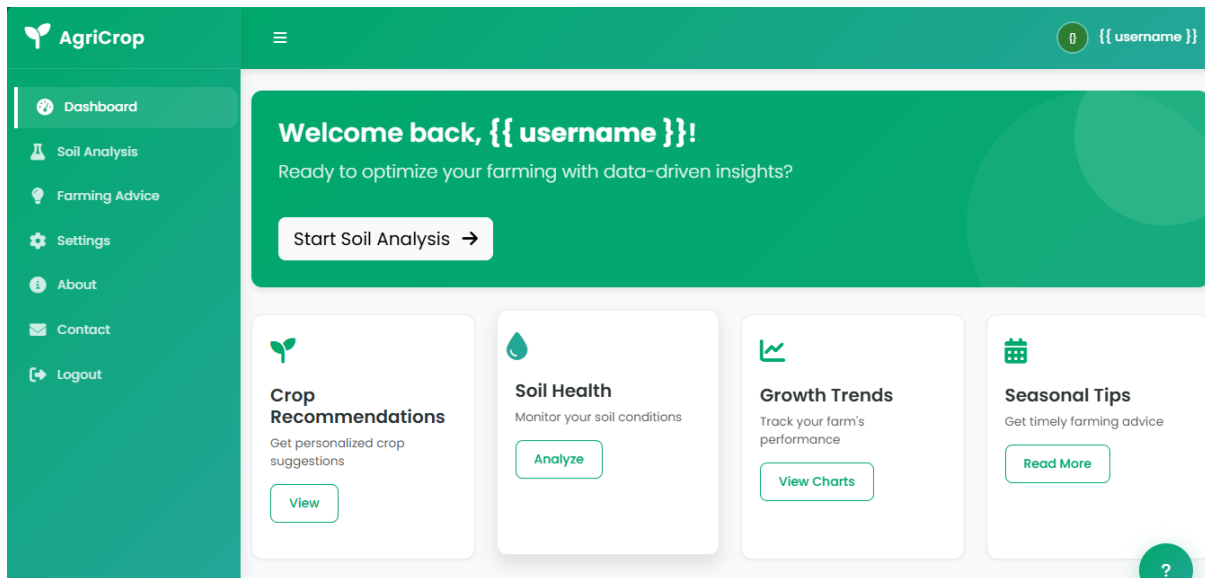


Figure 7. User Dashboard Unit

5.3 Results

5.3.1 Sensor Accuracy and Calibration

Calibration Against Standard Reference Materials: The NPK sensor was calibrated against water Components in Dew Drop as the Reference. The table below showed the reference and the measured value from NPK sensor and calibrated accordingly as shown clearly in Table. 4 below: -

Table 3. To show the measured value against the reference values

N (mg/kg)	P (mg/kg)	K (mg/kg)	pH	Moisture (%)	Temp (°C)	Reference Results
0	4	3.0	7.0	35	25	Measured Value
0	0	0.5	7.0	30	26.7	Expected value
±1	±4	±2.5	± 0	±5	±2	Calibrated value

Note that the annual rainfall data for Mbeya (214 mm/year) would be a static and constant input, used by the models to provide context, especially when recommending crops that are sensitive to water availability.

5.3.2 Testing Examples and its Results.

To demonstrate the system's capabilities, two example test scenarios were devised, simulating different soil conditions and observing the system's response as follow: -

5.3.2.1 Test Example of soil sample 0001.

Table 5. and Figure 8 tend to show the measured value, input values from the user(farmer) and recommended Crop and fertilizer as shown below: -

Table 4. Soil parameter of Soil Sample 0001 and its input values

N (mg/kg)	P (mg/kg)	K (mg/kg)	pH	Moisture (%)	Temp (°C)	Soil Type	Crop Grown	Rainfall (mm)
0	6	5	6.9	35	26.7	Sandy	Maize	214

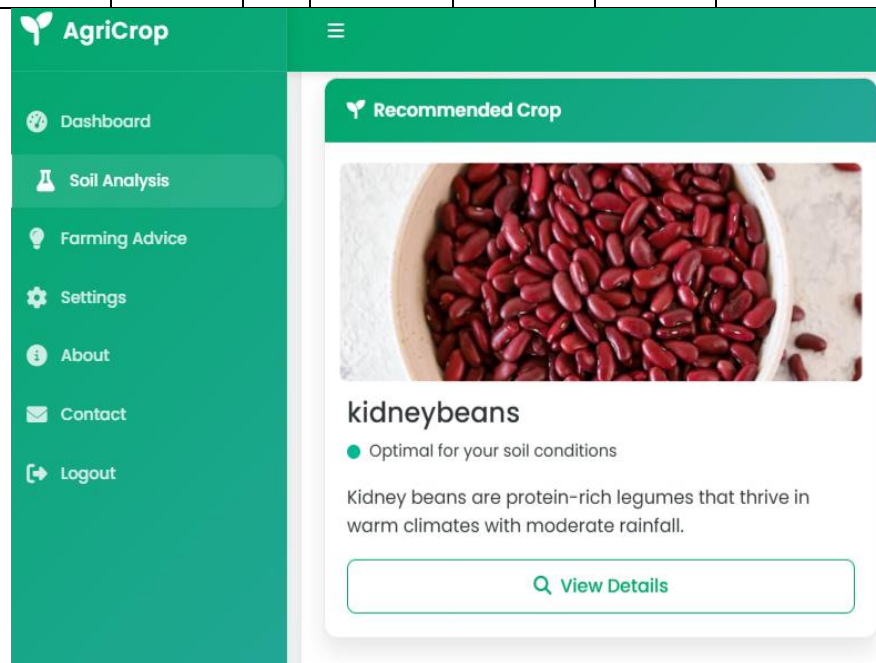


Figure 8. Recommended Crop of soil sample 0001

Table 5. and Figure 9 and Figure 10 tend to show the measured value, input values from the user(farmer) and recommended Crop and fertilizer as shown below: -

Table 5. Soil parameter of Soil Sample 0002 and its input values

N (mg/kg)	P (mg/kg)	K (mg/kg)	pH	Moisture (%)	Temp (°C)	Soil Type	Crop Grown	Rainfall (mm)
37	91	91	7.5	18.7	25	Loamy	Rice	214

Recommended Crop



chickpea

- Optimal for your soil conditions

Chickpea is a nutrient-rich legume that grows well in moderate climates with well-drained soil.

View Details

Figure 9. Recommended Crop of soil sample 0002

Recommended Fertilizer



Urea

- Balanced for your soil needs

Urea is a nitrogen-release fertilizer that promotes leafy growth. Apply at the beginning of the growing season.

View Details

Figure 10. Recommended fertilizer of soil sample 0002

5.4 Discussion of Results and Distinguishing Features

The system demonstrated strong potential in enhancing decision-making for farmers. By integrating real-time data and AI-driven models, farmers gained insights as follow: -

- ✓ **Enhanced Power Supply Reliability for Remote Operations:** A primary distinguishing feature of our system is its robust and reliable power supply mechanism. Unlike many reviewed systems that often assume constant AC power or rely on limited battery life, our design incorporates a rechargeable battery backup. This critical inclusion ensures continuous operation even in remote farm locations or areas with unreliable grid power.
- ✓ **Advanced User Interface for Deeper Insights:** A significant leap forward in user interaction is presented by our web-based application dashboard. While many comparable studies and existing solutions often rely on basic LCD displays for presenting recommendations, our project takes a substantial step ahead. The development of a well-designed, attractive, and intuitive website dashboard allows farmers to gain far greater insights into their farm's conditions. This includes not just current recommendations but also historical data visualization, trend analysis, and a more engaging interface, fostering a deeper understanding and empowering more informed decision-making beyond simple advisories.

Challenges Encountered:

Training Dataset Specificity: While the Random Forest models demonstrated strong performance using comprehensive datasets from Kaggle archives, the accuracy and compatibility of the recommendations could be further refined by incorporating more localized and experiential data. Specifically, the current training dataset lacks a significant volume of data from expert farmers in the Mbeya region, as well as specific data from local fertilizer organizations and agricultural institutions. Integrating such localized expertise and empirical data would enable the models to generate recommendations and advisories that are even more nuanced, accurate, and directly compatible with the unique soil characteristics, climatic patterns, and established agricultural practices within the target region. This is a critical area for future development to evolve the "machine" into a truly "more than human" advisory system, capable of reflecting the accumulated wisdom of local agricultural experience.

5.5 Summary

Through comprehensive technical, field, and user testing, the intelligent soil analyzer and crop fertilizer advisory system has proven its value as a transformative agricultural tool. By putting data and technology into the hands of farmers, we move towards a future where agriculture is more productive and sustainable.

CHAPTER 6: CONCLUSION AND RECOMMENDATIONS

This chapter summarizes the project's achievements and provides recommendations for further development and deployment.

6.1 Conclusion

This journey end here with archived objective of creating a system that could intelligently analyze soil conditions and provide tailored recommendations for crop selection and fertilizer application. Through the strategic deployment of an NPK sensor, capable of measuring nitrogen, phosphorus, potassium, temperature, pH, and moisture, we established a reliable pipeline for real-time soil data acquisition. The ESP8266 NodeMCU, with its Wi-Fi capabilities, proved to be an excellent choice for reliable transmitting the primary data to a local computer.

The intelligence of the system was archived through the careful training of two distinct Random Forest Classifier models, one for crop recommendation and another for fertilizer advisories. These models were trained on comprehensive secondary datasets sourced from the Kaggle archives, complemented by vital annual rainfall data for Mbeya obtained from the Tanzania Meteorological Agency (TMA). The Flask framework served as the backbone of our local computer system, managing data flow, interacting with the SQLite database for efficient storage. The frontend dashboard, built with HTML, CSS, and JavaScript, provides a user-friendly interface for farmers to visualize soil parameters and receive clear, data-backed recommendations.

Therefore, this system represents a significant step towards precision agriculture, offering farmers the ability to make informed decisions based on the actual needs of their soil. By optimizing crop selection and fertilizer use, the system holds important potential to enhance agricultural productivity, reduce resource wastage, and foster more sustainable farming practices in regions like Mbeya and other regions in Tanzania.

6.2 Recommendations

For Future Improvements:

- ✓ To improve calibration of the sensors to be more accuracy and precise so as to provide accurate data about the soil condition parameters.
- ✓ To provide another alternative power source supply like rechargeable batteries or solar panel so that the device can work any area especially in the farm where there is no Electrical power supply (AC Voltage) .

- ✓ To integrate local datasets like from TARI or Sokoine University of Agriculture. The external datasets from the platform like Datasets sometimes are not reality to the local area like Mbeya, So in training the models you needs to integrate these local data so that the system to be real and marketable to the most crops grown on the specific area.
- ✓ To integrate this system with another advanced Application Programming Interface (API) like OpenAI and Deepseek or Gamin so that to provide more advices and recommendations, this will allow farmer to get and derive more insight about the farm. Random forest classifier algorithm is a good recommending tool but it needs these API so that to create something with is Real and marketable product.

Therefore, the intelligent soil analyzer and crop fertilizer advisory system has proven to be a practical, cost-effective, and impactful tool for modern agriculture. It demonstrates how technology can support smallholder farmers in making informed, timely, and sustainable decisions. By continuing to refine the system and expand its data foundation, this tool has the potential to transform farming practices, improve food security, and promote sustainable farming practices.

REFERENCES

- A Decision Support System for Crop Recommendation Using Machine Learning Classification Algorithms.* (n.d.). Retrieved January 17, 2025, from <https://www.mdpi.com/2077-0472/14/8/1256>
- A, P., Chakraborty, S., Kumar, A., & Pooniwala, O. R. (2021). Intelligent Crop Recommendation System using Machine Learning. *2021 5th International Conference on Computing Methodologies and Communication (ICCMC)*, 843–848. <https://doi.org/10.1109/ICCMC51019.2021.9418375>
- Abinaya, P., Faaiza, A., Tharshni, S., Vishali, G., Aravindan, M., & Santhi, D. G. (2024). Survey on Soil Micro-Nutrients Analysis and Crop Recommendation System in Smart Agriculture. *International Research Journal on Advanced Engineering Hub (IRJAEH)*, 2(03), Article 03. <https://doi.org/10.47392/IRJAEH.2024.0066>
- Agricultural land suitability classification and crop suggestion using machine learning and spatial multicriteria decision analysis in semi-arid ecosystem | Environment, Development and Sustainability.* (n.d.). Retrieved January 17, 2025, from <https://link.springer.com/article/10.1007/s10668-023-04440-1>
- Akkem, Y., Biswas, S. K., & Varanasi, A. (2023). Smart farming using artificial intelligence: A review. *Engineering Applications of Artificial Intelligence*, 120, 105899. <https://doi.org/10.1016/j.engappai.2023.105899>
- Arduino uno wifi rev2.* (n.d.). Bing. Retrieved February 17, 2025, from <https://www.bing.com/images/search?q=arduino+uno+wifi+rev2&FORM=HDRSC3>
- Geypens, M., & Vandendriessche, H. (1996). Advisory systems for nitrogen fertilizer recommendations. In O. Van Cleemput, G. Hofman, & A. Vermoesen (Eds.), *Progress in Nitrogen Cycling Studies: Proceedings of the 8th Nitrogen Workshop held at the University of Ghent, 5–8 September, 1994* (pp. 329–336). Springer Netherlands. https://doi.org/10.1007/978-94-011-5450-5_55
- IoT-Driven Artificial Intelligence Technique for Fertilizer Recommendation Model | IEEE Journals & Magazine | IEEE Xplore.* (n.d.). Retrieved January 14, 2025, from <https://ieeexplore.ieee.org/abstract/document/9714817>
- IoT-Enabled Soil Nutrient Analysis and Crop Recommendation Model for Precision Agriculture.* (n.d.). Retrieved January 14, 2025, from <https://www.mdpi.com/2073-431X/12/3/61>
- IRJAEH+-+02-03-016+TO+WEBSITE.* (n.d.).

- Mishra, T. K., Mishra, S. K., Sai, K. J., Alekhya, B. S., & Nishith, A. R. (2021). Crop Recommendation System using KNN and Random Forest considering Indian Data set. *2021 19th OITS International Conference on Information Technology (OCIT)*, 308–312. <https://doi.org/10.1109/OCIT53463.2021.00068>
- Motamedi, B., & Villányi, B. (2024). A predictive analytics model with Bayesian-Optimized Ensemble Decision Trees for enhanced crop recommendation. *Decision Analytics Journal*, 12, 100516. <https://doi.org/10.1016/j.dajour.2024.100516>
- Motwani, A., Patil, P., Nagaria, V., Verma, S., & Ghane, S. (2022). Soil Analysis and Crop Recommendation using Machine Learning. *2022 International Conference for Advancement in Technology (ICONAT)*, 1–7. <https://doi.org/10.1109/ICONAT53423.2022.9725901>
- Pudumalar, S., Ramanujam, E., Rajashree, R. H., Kavya, C., Kiruthika, T., & Nisha, J. (2017). Crop recommendation system for precision agriculture. *2016 Eighth International Conference on Advanced Computing (ICoAC)*, 32–36. <https://doi.org/10.1109/ICoAC.2017.7951740>
- Rahman, Md. J., Ahmed, Md. S., Biswas, S., Orchi, A. T., Rahman, R., & Islam, A. K. M. M. (2024). Cc. *2024 6th International Conference on Electrical Engineering and Information & Communication Technology (ICEEICT)*, 1356–1361. <https://doi.org/10.1109/ICEEICT62016.2024.10534582>
- Rajak, R. K., Pawar, A., Pendke, M., Shinde, P., Rathod, S., & Devare, A. (n.d.). *Crop Recommendation System to Maximize Crop Yield using Machine Learning Technique*. 04(12).
- Sankareswari, K., & Sujatha, G. (2024). Crop and Fertiliser Recommendation System for Sustainable Agricultural Development. In S. Balasubramanian, G. Natarajan, & P. R. Chelliah (Eds.), *Intelligent Robots and Drones for Precision Agriculture* (pp. 327–349). Springer Nature Switzerland. https://doi.org/10.1007/978-3-031-51195-0_16

APPENDIX A. SYSTEM SOFTWARE DOCUMENTATION

Source Code Documentation: Intelligent Soil Analyzer and Crop Fertilizers Advisory System

This documentation provides a step-by-step guide for setting up the environment, understanding the project structure, and running the "Intelligent Soil Analyzer and Crop Fertilizers Advisory System" from its source code. This guide is intended for developers and users who wish to deploy or further develop the system.

1. System Overview

The "Intelligent Soil Analyzer and Crop Fertilizers Advisory System" is a Flask-based web application that provides farmers with intelligent recommendations for crop selection and fertilizer application based on real-time soil data. It integrates hardware (NPK sensor, ESP8266 NodeMCU) for data acquisition and machine learning models (Random Forest Classifiers) for generating advisories.

Key Features:

1. Real-time soil NPK, temperature, pH, and moisture data collection.
2. Wireless data transmission from sensor to local computer.
3. Machine learning models for crop and fertilizer recommendations.
4. Web-based user dashboard for data visualization and recommendations.
5. SQLite database for data storage.

2. Prerequisites

Before setting up the project, ensure you have the following software and hardware components:

2.1 Software Requirements

1. Python 3.8+: The primary programming language for the backend.
2. pip: Python's package installer (usually comes with Python).
3. Git (Optional but Recommended): For cloning the repository.
4. Arduino IDE (or PlatformIO): For programming the ESP8266 NodeMCU.

2.2 Python Libraries

The Python backend requires the following libraries:

```
Flask
werkzeug (for security utilities, usually installed with Flask)
sqlite3 (built-in with Python)
pickle (built-in with Python)
requests
scikit-learn (for the machine learning models)
```

2.3 Hardware Requirements

1. Local Computer: A desktop or laptop to run the Flask application and host the database.
2. ESP8266 NodeMCU and MAX485: A Wi-Fi enabled microcontroller to interface with the sensor and transmit data.
3. NPK Sensor (7-in-1): A soil sensor capable of measuring Nitrogen, Phosphorus, Potassium, Temperature, pH, and Moisture. Ensure it has a compatible interface (e.g., UART/RS485).
4. Wi-Fi Network: Both the ESP8266 and the local computer must be connected to the same Wi-Fi network for data transmission (AgriCrop Network).
5. Power Supply for ESP8266/Sensor: A reliable power source for the sensor unit, ideally with a backup battery for remote operations.

2.4 Model Files

The project relies on pre-trained machine learning models. You will need:

Crop.pkl: The pickled Random Forest Classifier model for crop recommendations, along with its associated LabelEncoder (or similar encoder for inverse transformation).

fertilizer.pkl: The pickled Random Forest Classifier model for fertilizer recommendations, along with its associated LabelEncoder for fertilizer types and mappings for soil_types and crop_types.

- Note: These .pkl files are generated during the model training phase (which is outside the scope of this main.py file). You must ensure these files are correctly placed in the model/ directory as described in the project structure. If you do not have these, the application will fail to start.

3. Project Structure

The project expects a specific directory structure for its components to function correctly.

your_project_root/

```
|— main.py
|— model/
|   |— Crop.pkl
|   |— fertilizer.pkl
|— templates/
|   |— login.html
|   |— register.html    (if separate, or integrated into login.html)
|   |— home.html
|   |— sensor_data.html
|   |— ...              (any other HTML templates)
|— static/
|   |— css/
|   |   |— style.css    (or similar)
|   |— js/
|   |   |— script.js    (or similar)
|   |— images/
|       |— crops/
|       |   |— rice.jpg
|       |   |— maize.jpg
|       |   |— ...      (other crop images)
|       |— fertilizers/
|           |— urea.jpg
|           |— dap.jpg
|           |— ...      (other fertilizer images)
|— system.db            (will be created automatically on first run)
```

The model/ directory must exist and contain Crop.pkl and fertilizer.pkl.

The templates/ directory must contain all HTML files rendered by Flask.

The static/ directory must contain CSS, JavaScript, and image assets. Ensure the image paths in main.py (/static/images/crops/ and /static/images/fertilizers/) correctly point to your image files.

4. Setup Instructions

Follow these steps to set up the project environment.

4.1 Clone the Repository (or Download Files)

If your project is in a Git repository, clone it:

```
git clone <your-repository-url> cd your_project_root
```

Otherwise, download all project files and place them in a directory named your_project_root.

4.2 Create a Python Virtual Environment

It's highly recommended to use a virtual environment to manage project dependencies.

```
python -m venv venv
```

4.3 Activate the Virtual Environment

On Windows:

```
.\venv\Scripts\activate
```

On macOS/Linux:

```
source venv/bin/activate
```

- ✓ You should see (venv) prepended to your terminal prompt, indicating the virtual environment is active.

4.4 Install Python Dependencies

Create a requirements.txt file in your your_project_root directory with the following content:

```
Flask scikit-learn requests werkzeug
```

Then, install them using pip:

```
pip install -r requirements.txt
```

4.5 Place Model Files

Ensure your Crop.pkl and fertilizer.pkl files are placed inside the model/ directory within your project root. If this directory doesn't exist, create it.

4.6 Prepare ESP8266 NodeMCU

This is a crucial step that requires separate programming of the ESP8266.

Connect NPK Sensor to ESP8266: Wire your 7-in-1 NPK sensor to the ESP8266 NodeMCU according to the sensor's datasheet and the ESP8266's pinout. Typically, NPK sensors communicate via RS485, requiring an RS485-to-TTL converter module to interface with the ESP8266's UART pins.

Program ESP8266:

1. Using Arduino IDE (or PlatformIO), write and upload firmware to the ESP8266.
2. The firmware must:
3. Connect to your local Wi-Fi network (the same network your computer is on).
4. Periodically read data from the NPK sensor.
5. Host a simple web server (e.g., using ESP8266WebServer library).
6. Expose an endpoint (e.g., /data) that, when accessed via HTTP GET, returns the latest sensor readings in JSON format.

Example JSON output from /data endpoint:

```
{ "nitrogen": 75.0, "phosphorus": 22.0, "potassium": 110.0, "temperature": 28.5, "moisture": 65.0, "ph": 6.8, "rainfall": 214.0 }
```

Configure the ESP8266 to have a static IP address or ensure you know its assigned IP address. The main.py file uses ESP_AP_IP = "192.168.4.1" by default. If your ESP8266 has a different IP, you must update this line in main.py to match your ESP8266's IP address.

5. Running the Application

Once all prerequisites and setup steps are complete, you can run the Flask application.

5.1 Initialize the Database

The init_db() function in main.py will create the necessary SQLite database tables (system.db) if they don't already exist. This happens automatically when main.py is run.

5.2 Start the Flask Application

Navigate to your your_project_root directory in the terminal (with your virtual environment activated) and run:

```
python main.py
```

You should see output indicating that the Flask development server is running, usually on http://127.0.0.1:5000/. You will also see messages about model loading and the sensor polling thread starting.

Note on use_reloader=False: The main.py uses app.run(debug=True, use_reloader=False). use_reloader=False prevents the Flask development server from restarting twice, which is important because the sensor_thread is started directly in main.py and would otherwise be

uplicated. For production, debug=False should be used, and a production-grade WSGI server (like Gunicorn or uWSGI) should be employed.

5.3 Access the Web Dashboard

Open your web browser and go to the address provided by Flask, typically: `http://127.0.0.1:5000/`

6. Using the Application

6.1 User Authentication

Register: If you are a new user, navigate to the registration page (often linked from the login page or directly via /register if implemented as a separate route). Provide a unique username, email, and password.

Login: Use your registered credentials to log in.

6.2 Dashboard Overview

1. After logging in, you will be redirected to the home dashboard. Here, you can view:
2. Your recent soil data entries.
3. Latest recommendations.
4. Overall statistics (total analyses, crops, fertilizers).
5. The very latest sensor data reading (if the sensor polling thread is active).

APPENDIX B. SYSTEM SOURCE CODES

ESP8266 NODE MCU CODES

```
#include <SoftwareSerial.h>

// RS485 connections

const byte ROPin = 14; // D5 = GPIO14 (RX)
const byte DIPin = 12; // D6 = GPIO12 (TX)
const byte DE_RE = 13; // D7 = GPIO13 (Direction)

SoftwareSerial max485Serial(ROPin, DIPin); // RX, TX

// Correct register addresses for 7-in-1 NPK sensor

const uint16_t REG_TEMP = 0x0000;
const uint16_t REG_MOIS = 0x0001;
const uint16_t REG_EC    = 0x0002;
const uint16_t REG_PH    = 0x0003;
const uint16_t REG_N      = 0x0004;
const uint16_t REG_P      = 0x0005;
const uint16_t REG_K      = 0x0006;

byte response[7];

void setup() {
    Serial.begin(9600);
    max485Serial.begin(4800);
    pinMode(DE_RE, OUTPUT);
    digitalWrite(DE_RE, LOW); // Start in receive mode
    Serial.println("\nESP8266 7-in-1 Soil Sensor Initialized");
    delay(1000);
}

void loop() {
    float temperature = readSensor(REG_TEMP) / 10.0;
    float moisture     = readSensor(REG_MOIS) / 10.0;
    uint16_t ec        = readSensor(REG_EC);
    float ph           = readSensor(REG_PH) / 10.0;
    uint16_t nitrogen  = readSensor(REG_N);
    uint16_t phosphor  = readSensor(REG_P);
    uint16_t potass    = readSensor(REG_K);

    Serial.println("\n=== Sensor Readings ===");
    Serial.printf("Temperature: %.1f °C\n", temperature);
    Serial.printf("Moisture: %.1f %%\n", moisture);
    Serial.printf("EC: %d µS/cm\n", ec);
    Serial.printf("pH: %.1f\n", ph);
    Serial.printf("Nitrogen: %d mg/kg\n", nitrogen);
    Serial.printf("Phosphorous: %d mg/kg\n", phosphor);
}
```

```

Serial.printf("Potassium: %d mg/kg\n", potass);

delay(3000); // Wait before next reading
}

uint16_t readSensor(uint16_t regAddress) {
    clearReceiveBuffer();

    // Build Modbus request: [Device Addr][Function][Start reg Hi][Start reg Lo][Qty Hi][Qty
    Lo][CRC Lo][CRC Hi]

    byte request[8];

    request[0] = 0x01;      // Device address
    request[1] = 0x03;      // Function code: Read Holding Registers
    request[2] = regAddress >> 8; // Start address high byte
    request[3] = regAddress & 0xFF; // Start address low byte
    request[4] = 0x00;      // Quantity high byte
    request[5] = 0x01;      // Quantity low byte (read 1 register)

    uint16_t crc = calculateCRC(request, 6);
    request[6] = crc & 0xFF;      // CRC low byte
    request[7] = crc >> 8;      // CRC high byte

    // Enable transmit
    digitalWrite(DE_RE, HIGH);
    delayMicroseconds(100);
    max485Serial.write(request, 8);
    max485Serial.flush();
    // Switch to receive
    digitalWrite(DE_RE, LOW);
    delay(2); // Allow bus to settle
    // Wait for response
    unsigned long startTime = millis();
    int index = 0;
    while (millis() - startTime < 500) {
        if (max485Serial.available()) {
            response[index++] = max485Serial.read();
            if (index >= 7) break; // Expected response length for 1 register
        }
    }

    // Print raw response for debugging
    Serial.print("Response: ");
    for (int i = 0; i < index; i++) {

```

```

        if (response[i] < 0x10) Serial.print("0");
        Serial.print(response[i], HEX);
        Serial.print(" ");
    }
    Serial.println();
    // Check if we received enough bytes
    if (index < 7) {
        Serial.println("Error: Incomplete response");
        return 0;
    }
    // Combine data bytes into one 16-bit value
    uint16_t value = (response[3] << 8) | response[4];
    return value;
}

void clearReceiveBuffer() {
    while (max485Serial.available()) {
        max485Serial.read();
    }
}

// Modbus RTU CRC16 calculation
uint16_t calculateCRC(byte *data, uint8_t length) {
    uint16_t crc = 0xFFFF;
    for (uint8_t i = 0; i < length; i++) {
        crc ^= data[i];
        for (uint8_t j = 0; j < 8; j++) {
            if (crc & 0x0001)
                crc = (crc >> 1) ^ 0xA001;
            else
                crc >>= 1;
        }
    }
    return crc;
}

```

BACKEND PYTHON FLASK CODES

```
from flask import Flask, render_template, request, redirect, url_for,
session, flash, jsonify

from werkzeug.security import generate_password_hash, check_password_hash

import sqlite3

import pickle

import os

from werkzeug.utils import secure_filename

from datetime import datetime

import database

app = Flask(__name__)

app.secret_key = 'your_secret_key_here'

# 1. Get the project's root directory

BASE_DIR = os.path.dirname(os.path.abspath(__file__))


# 2. Define paths to your model files

MODEL_DIR = os.path.join(BASE_DIR, 'model') # Folder named "model"

CROP_MODEL_PATH = os.path.join(MODEL_DIR, 'Crop.pkl')

FERTILIZER_MODEL_PATH = os.path.join(MODEL_DIR, 'fertilizer.pkl')

# 3. Print paths for debugging (remove after confirmation)

print(f"Looking for Crop model at: {CROP_MODEL_PATH}")

print(f"Looking for Fertilizer model at: {FERTILIZER_MODEL_PATH}")


# 4. Load models with verification

def load_model(model_path):

    """Safe model loader with validation"""

    if not os.path.exists(model_path):

        raise FileNotFoundError(f"Model file missing at: {model_path}")


    with open(model_path, 'rb') as f:

        model = pickle.load(f)

        if not hasattr(model, 'predict'):

            raise ValueError(f"File {model_path} doesn't contain a valid
scikit-learn model")

        return model
```

```

try:
    crop_model = load_model(CROP_MODEL_PATH)
    fertilizer_model = load_model(FERTILIZER_MODEL_PATH)

    print("✓ Models loaded successfully")
except Exception as e:
    print(f"✗ Model loading failed: {str(e)}")
    # Exit if models are essential
    exit(1)

# Database connection helper
def get_db_connection():
    conn = sqlite3.connect('system.db')
    conn.row_factory = sqlite3.Row
    return conn

# Homepage route
@app.route('/')
def index():
    if 'user_id' not in session:
        return redirect(url_for('login'))
    return redirect(url_for('home'))

# Login route
@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        username = request.form['username']
        password = request.form['password']

        conn = get_db_connection()
        user = conn.execute('SELECT * FROM users WHERE username = ?',
                             (username,)).fetchone()
        conn.close()

        if user and check_password_hash(user['password'], password):
            session['user_id'] = user['id']
            session['username'] = user['username']

```

```

        flash('Login successful!', 'success')
        return redirect(url_for('home'))
    else:
        flash('Invalid username or password', 'danger')

    return render_template('login.html')

# Registration route
@app.route('/register', methods=['POST'])
def register():
    username = request.form['username']
    password = request.form['password']
    email = request.form['email']

    hashed_password = generate_password_hash(password)

    try:
        conn = get_db_connection()
        conn.execute('INSERT INTO users (username, password, email) VALUES
(?, ?, ?)',
                    (username, hashed_password, email))
        conn.commit()
        conn.close()
        return redirect(url_for('login', registered='true'))
    except sqlite3.IntegrityError:
        flash('Username or email already exists', 'danger')
        return redirect(url_for('login'))

# Logout route
@app.route('/logout')
def logout():
    session.clear()
    flash('You have been logged out', 'info')
    return redirect(url_for('login'))

```



```

# Home route
@app.route('/home')
def home():
    if 'user_id' not in session:
        return redirect(url_for('login'))

    conn = get_db_connection()

    # Get user's recent soil data entries
    soil_data = conn.execute('''
        SELECT * FROM soil_data
        WHERE user_id = ?
        ORDER BY recorded_at DESC
        LIMIT 5
    ''', (session['user_id'],)).fetchall()

    # Get recent recommendations
    recommendations = conn.execute('''
        SELECT r.*, s.recorded_at as soil_data_date
        FROM recommendations r
        JOIN soil_data s ON r.soil_data_id = s.id
        WHERE s.user_id = ?
        ORDER BY r.generated_at DESC
        LIMIT 5
    ''', (session['user_id'],)).fetchall()

    # Get stats for dashboard
    total_analyses = conn.execute('SELECT COUNT(*) FROM soil_data WHERE
user_id = ?',
                                (session['user_id'],)).fetchone()[0]

    total_crops = conn.execute('SELECT COUNT(DISTINCT recommended_crop)
FROM recommendations r JOIN soil_data s ON r.soil_data_id = s.id WHERE
s.user_id = ?',
                                (session['user_id'],)).fetchone()[0]

```

```

        total_fertilizers = conn.execute('SELECT COUNT(DISTINCT
recommended_fertilizer) FROM recommendations r JOIN soil_data s ON
r.soil_data_id = s.id WHERE s.user_id = ?',
                                         (session['user_id'],)).fetchone()[0]

    conn.close()

    return render_template('home.html',
                           username=session['username'],
                           soil_data=soil_data,
                           recommendations=recommendations,
                           total_analyses=total_analyses,
                           total_crops=total_crops,
                           total_fertilizers=total_fertilizers)

# Process soil data and make recommendations
@app.route('/analyze', methods=['POST'])
def analyze():
    if 'user_id' not in session:
        return jsonify({'error': 'Unauthorized'}), 401

    try:
        # Get form data with proper error handling
        try:
            data = {
                'nitrogen': float(request.form.get('nitrogen', 0)),
                'phosphorus': float(request.form.get('phosphorus', 0)),
                'potassium': float(request.form.get('potassium', 0)),
                'ph': float(request.form.get('ph', 7)),
                'moisture': float(request.form.get('moisture', 0)),
                'temperature': float(request.form.get('temperature', 25)),
                'rainfall': float(request.form.get('rainfall', 0)),
                'soil_type': request.form.get('soil_type', 'loamy'),
                'crop_grown': request.form.get('crop_grown', '')
            }

```

```

except ValueError as e:
    return jsonify({'error': f'Invalid input data: {str(e)}'}), 400

# Save soil data to database
conn = None
try:
    conn = get_db_connection()
    cursor = conn.cursor()
    cursor.execute('''
        INSERT INTO soil_data
            (user_id, nitrogen, phosphorus, potassium, ph, moisture,
temperature, rainfall, soil_type, crop_grown)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
        ''', (session['user_id'], data['nitrogen'], data['phosphorus'],
data['potassium'],
            data['ph'], data['moisture'], data['temperature'],
data['rainfall'],
            data['soil_type'], data['crop_grown']))

    soil_data_id = cursor.lastrowid

# Prepare data for ML models with fallback values
crop_ml_data = [
    data['nitrogen'],
    data['phosphorus'],
    data['potassium'],
    data['temperature'],
    data['moisture'],
    data['ph'],
    data['rainfall']
]

# Convert categorical data to numerical for fertilizer model
soil_type_map = {'sandy': 0, 'loamy': 1, 'clay': 2, 'silty': 3,
'peaty': 4, 'chalky': 5}

```

```

        crop_type_map = {'': 0, 'rice': 1, 'wheat': 2, 'maize': 3,
'soybean': 4, 'cotton': 5}

        fertilizer_ml_data = [
            data['temperature'],
            data['moisture'],
            soil_type_map.get(data['soil_type'].lower(), 1), # Default
to loamy
            crop_type_map.get(data['crop_grown'].lower(), 0), #
Default to none
            data['nitrogen'],
            data['phosphorus'],
            data['potassium']
        ]

        # Get recommendations with model error handling
        try:
            recommended_crop =
str(crop_model.predict([crop_ml_data])[0])
            recommended_fertilizer =
str(fertilizer_model.predict([fertilizer_ml_data])[0])
        except Exception as e:
            return jsonify({'error': f'Model prediction failed:
{str(e)}'}), 500

        # Get additional info
        crop_info = get_crop_info(recommended_crop)
        fertilizer_info = get_fertilizer_info(recommended_fertilizer)

        # Save recommendation
        cursor.execute('''
            INSERT INTO recommendations
            (soil_data_id, recommended_crop, recommended_fertilizer,
            crop_description, fertilizer_description, crop_image_path,
fertilizer_image_path)
            VALUES (?, ?, ?, ?, ?, ?, ?)
        ''', (soil_data_id, recommended_crop, recommended_fertilizer,

```

```

        crop_info['description'], fertilizer_info['description'],
        crop_info['image_path'], fertilizer_info['image_path']))

    recommendation_id = cursor.lastrowid

    conn.commit()

    # Get the new recommendation

    recommendation = conn.execute('SELECT * FROM recommendations
WHERE id = ?', (recommendation_id,)).fetchone()

    return jsonify({
        'success': True,
        'recommendation': dict(recommendation)
    })

except sqlite3.Error as e:
    return jsonify({'error': f'Database error: {str(e)}'}), 500
finally:
    if conn:
        conn.close()

except Exception as e:
    return jsonify({'error': f'Server error: {str(e)}'}), 500
# Helper functions to get crop and fertilizer info
def get_crop_info(crop_name):
    crop_descriptions = {
        'rice': {
            'description': 'Rice is a staple food for over half the world
population. It grows best in warm, humid climates with plenty of water.',
            'image_path': '/static/images/crops/rice.jpg'
        },
        'wheat': {
            'description': 'Wheat is a cereal grain that is grown
worldwide. It prefers temperate climates and well-drained soil.',
            'image_path': '/static/images/crops/wheat.jpg'
        },

```

```

        'maize': {
            'description': 'Maize (corn) is a versatile crop used for food,
feed, and industrial products. It requires warm temperatures and moderate
rainfall.',
            'image_path': '/static/images/crops/maize.jpg'
        },
        'default': {
            'description': 'This crop is well-suited to your soil
conditions. Consult local agricultural experts for specific growing
advice.',
            'image_path': '/static/images/crops/default.jpg'
        }
    }

    return crop_descriptions.get(crop_name.lower(),
crop_descriptions['default'])

def get_fertilizer_info(fertilizer_name):
    fertilizer_descriptions = {
        'urea': {
            'description': 'Urea is a nitrogen-release fertilizer that
promotes leafy growth. Apply at the beginning of the growing season.',
            'image_path': '/static/images/fertilizers/urea.jpg'
        },
        'dap': {
            'description': 'Diammonium Phosphate (DAP) provides both
nitrogen and phosphorus. Good for early plant growth and root
development.',
            'image_path': '/static/images/fertilizers/dap.jpg'
        },
        'mop': {
            'description': 'Muriate of Potash (MOP) is a potassium-rich
fertilizer that improves fruit quality and disease resistance.',
            'image_path': '/static/images/fertilizers/mop.jpg'
        },
        'default': {
            'description': 'This fertilizer is recommended for your soil
conditions. Follow package instructions for application rates.',
            'image_path': '/static/images/fertilizers/default.jpg'
        }
    }

```

```
        }

    }

    return fertilizer_descriptions.get(fertilizer_name.lower(),
fertilizer_descriptions['default'])

if __name__ == '__main__':
    # Initialize database
    database.init_db()
    app.run(debug=True, use_reloader=False)
```